DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

OBJECT ORIENTED PROGRAMMING

Course Code: CT-260

ASSIGNMENT — DESIGN PATTERNS IN REAL-WORLD SOFTWARE SYSTEMS

Semester	Spring 2026
Class	1st Year — BS Computer Science & Information Technology
Instructor	Dr. Murk Marvi
Total Marks	5 Marks
Submission Deadline	30 Days
Assignment Type	Group Assignment, 3 members in a group

PROJECT

CONTENT DELIVERY NETWORK (CDN)

GROUP MEMBERS

Ali Hasan Farooqui CT-25098
Areeba CT-25070

Task A – Application Overview

What does the application do?

A CDN is a network system made up of many servers spread across different locations all over the globe. The main purpose of a CDN ios is to deliver content to the end users in the fastest way possible.

When a user opens a website rather than their request traveling all the way to a central server (Origin Server) located on the other side of the world the CDN sends the request to a server located near to the user (Edge Server). This makes the website open much faster.

So examples of CDNs are CLoudflare and Akamai. These are used by millions of websites including Youtube and Netflix.

Who are its Users:

- Website owners and Business. The companies want their websites to be loaded fast. They use CDN on their content
- The End Users. These are the regular people who visit the website or stream videos online. The CDN makes their pages/videos load faster.

Core Features and Modules:

1. ***Origin Server:*** The main server that stores the original copy of all the website's content.

2. ***Edge Server:*** Servers placed in many different cities and countries. These store copies of the content close to users.

3. ***Cache*****:** A temporary storage on each edge server. When a user requests a file, the edge server checks if it already has a saved copy (cache hit) or needs to fetch it from the origin server (cache miss).

4. ***Request Router*** **:** The part of the system that decides which edge server should handle each incoming user request.

Why We chose this Application:

A CDN is a great real world application which simply moves the content physically closer to a user. We chose a CDN because all the other groups picked the obvious choices like a food delivery app or an online application. Those are fine in their own way but every pattern in those is pretty straightforward. U order u pay and then u are notified. Done

A CDN on the other hand solves problems which is exactly
what design patterns were invented for. Caching is a proxy problem by definition. A CDN has to have a configuration to be consistent everywhere which is a singleton problem by definition. Servers that need to react when something changes - Observer pattern. The application and patterns fit so perfectly together as if these CDN's were designed around GoF patterns.

We use CDN's everyday without even knowing it . Every video we watch online, every website we open there exists a CDN behind it.

Task B – Pattern Identification and Justification

Pattern 1 — Proxy:

Where it applies:
The proxy pattern is used in the Content Delivery Network, where the Edge Server is used as a proxy server to the Origin Server. When the user wants to download a file, like images, videos, or web pages, the request goes to the Edge Server first, not directly to the Origin Server. The user "thinks" they are talking directly to the source (Origin Server) but they are actually talking to a representative (Edge Server) .

If the requested file is available in the cache memory of the Edge Server, then it will be sent directly to the user. If the requested file is not available in the cache memory, then the edge server will send the request to the origin server, retrieve the requested file, store it in its cache for future requests and then send it to the user.

Why Proxy:
The Proxy pattern is used to protect an object from external access. In this case, the edge server is used as a proxy server to the origin server. It helps in improving the performance by providing frequently requested files. Without a proxy every single user request would have to travel to the Origin Server which may have been located on the other side of the world. Additionally thousands of users requesting from the same Origin Server would cause immense load further slowing down the network

OOP Principles reinforced:

1. Abstraction
2. Encapsulation
3. Inheritance

Pattern 2 — Iterator:

Where it applies:
Each Edge Server contains a cache list of hundreds or thousand of files. The server needs to perform a routine check for expired files or clearing out old data to make room for new. Instead of allowing every part of the server to access to the private list, the Edge Server provides an Iterator. The Iterator allows the system to check each file regardless of it being stored as a vector, linked list or a queue.

Why Iterator:
Without the Iterator if we changed the Edge Servers cache from a vector list to a more complex data structure like a linked list we would have to rewrite every part of the code again . The Iterator creates a standard interface for traversal.

OOP Principles reinforced:

1. Abstraction
2. Encapsulation
3. Decoupling

Pattern 3 — Observer:

Where it applies:
In a CDN the Origin Server holds the original file. It also maintains a list of all the Edge Servers that have cached copies of that original file .
For ex: if the content changes in the Origin Server, all the CDN Edge Servers should be notified to update their cache accordingly.

The Origin Server is the subject, and the CDN Edge Servers are the observers that need to be notified..

Why Observer:
Observer pattern is the behavioral pattern that allows a single subject to be observed by multiple objects. Automatically notifying the state of the subject changes. The main aim from this pattern is to avoid the Edge Servers constantly asking the Origin Server if there is an updated version of the files that they have cached. This uses unnecessary bandwidth and adds loads to the network.

The observer pattern also allows us to easily add a new Edge Server to the Origin Server's list. Whenever a new Edge Server is created it is automatically added to the list and is notified whenever there is a change along with every other Edge Server on the network

OOP Principles reinforced:

4. Polymorphism
5. Abstraction
6. Inheritance

Pattern 4— Strategy:

Where it applies:

This pattern is applied in the working of request routers. The request router which is the brain of the whole network needs to decide which edge server to pick to handle the user's request. The request router makes the decision based on 2 factors, the location of the closest edge server to the user and the least loaded edge server available. The request router might use a "Geographic Proximity" algorithm today, picking the server physically closest to the user but tomorrow it might want to switch to "Least Loaded Server" to avoid sending too much traffic to one busy server.

The strategy pattern allows the request router to swap these two algorithms any time it wants. Each algorithm is written as its own separate class, and the request router simply calls whichever strategy is currently alive.

Why Strategy:

Without strategy the routing logic would be nothing but multiple if and else conditions. Everytime we would have wanted to add a new route to the routing table we would have had to modify the request router class itself. This is risky and inefficient. Strategy solves this problem by separating each algorithm by giving it its own class so that they can be added or changed independently.

OOP Principles reinforced:

1. Abstraction
2. Encapsulation
3. Polymorphism
4. Inheritance

Task C – C++ Implementation

Implementation 1 — Proxy Pattern (Edge Server Caching)

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Server{
      public:
            virtual string requestContent(string fileName) = 0;
            virtual ~Server(){}
};
class OriginServer : public Server{
      public:
            string requestContent(string filename) override {
                  return "Data of (Origin Server)" + filename;
            }
};

class EdgeServer : public Server{
      private:
            OriginServer* origin;
            vector<string> cache;

            bool findInCache(string filename){
                  for(int i=0; i<cache.size(); i++){
                        if(cache[i] == filename){
                              return true;
                        }
                  }
                  return false;
            }
      public:
            EdgeServer(OriginServer* o){
                 origin = o;
            }
            string requestContent(string filename){
                  if(findInCache(filename)){
                        return "Data from Edge Cache: " + filename;
                  }0
                  else {
                        cache.push_back(filename);
                        return origin->requestContent(filename);
```

```
            }
        }
        void addFile(string filename){
            cache.push_back(filename);
        }
};
int main() {
    OriginServer origin;
    EdgeServer edge(&origin);

    edge.addFile("file1.txt");
    edge.addFile("file2.txt");

    cout << edge.requestContent("file1.txt") << endl;
    cout << edge.requestContent("file3.txt") << endl;
    cout << edge.requestContent("file2.txt") << endl;
    cout << edge.requestContent("file3.txt") << endl;

    return 0;
}
```

Implementation 2 — Iterator Pattern

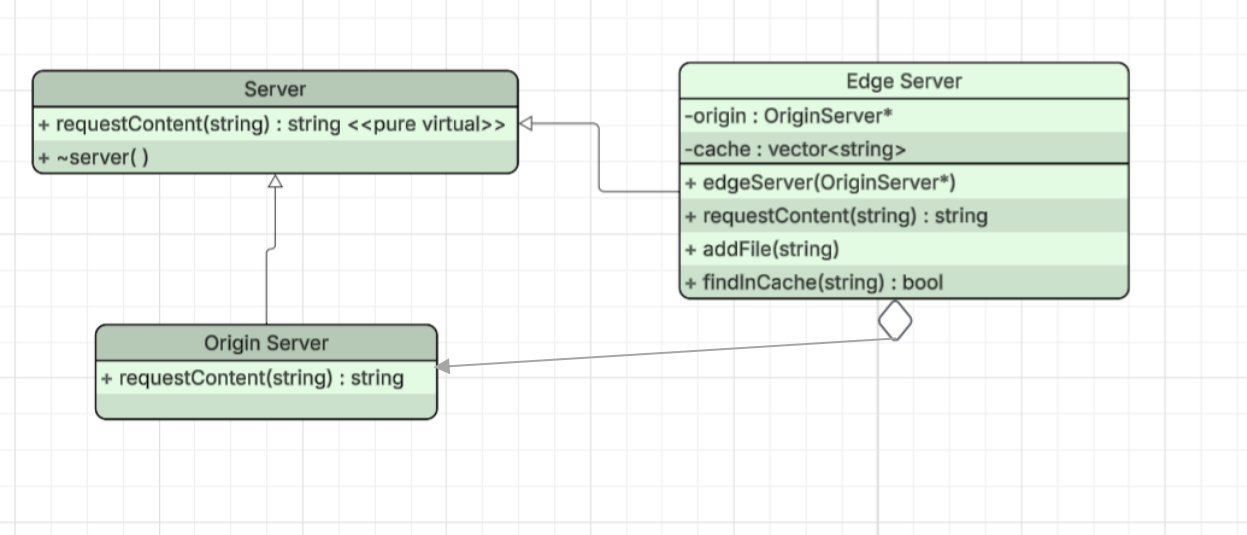
```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Iterator {
public:
    virtual bool hasNext() = 0;
    virtual string next() = 0;
    virtual ~Iterator() {}
};

class CacheIterator : public Iterator {
private:
    vector<string> cache;
    int index;

public:
    CacheIterator(const vector<string>& files) {
        cache = files;
```

```
        index = 0;
    }

    bool hasNext() override {
        return index < cache.size();
    }

    string next() override {
        return cache[index++];
    }
};

class EdgeServer {
private:
    vector<string> cache;
    string name;
public:
    EdgeServer(string n) : name(n) {}

    void addFile(string filename) {
        cache.push_back(filename);
        cout << name << " added file: " << filename << endl;
    }

    CacheIterator createIterator() {
        return CacheIterator(cache);
    }

    string getName() {
        return name;
    }
};

int main() {
    EdgeServer edge("Edge-Server-1");
    edge.addFile("video.mp4");
    edge.addFile("image.jpg");
    edge.addFile("style.css");
    edge.addFile("script.js");

    cout << "\nIterating through cached files: \n";

    CacheIterator it = edge.createIterator();

    while(it.hasNext()) {
```

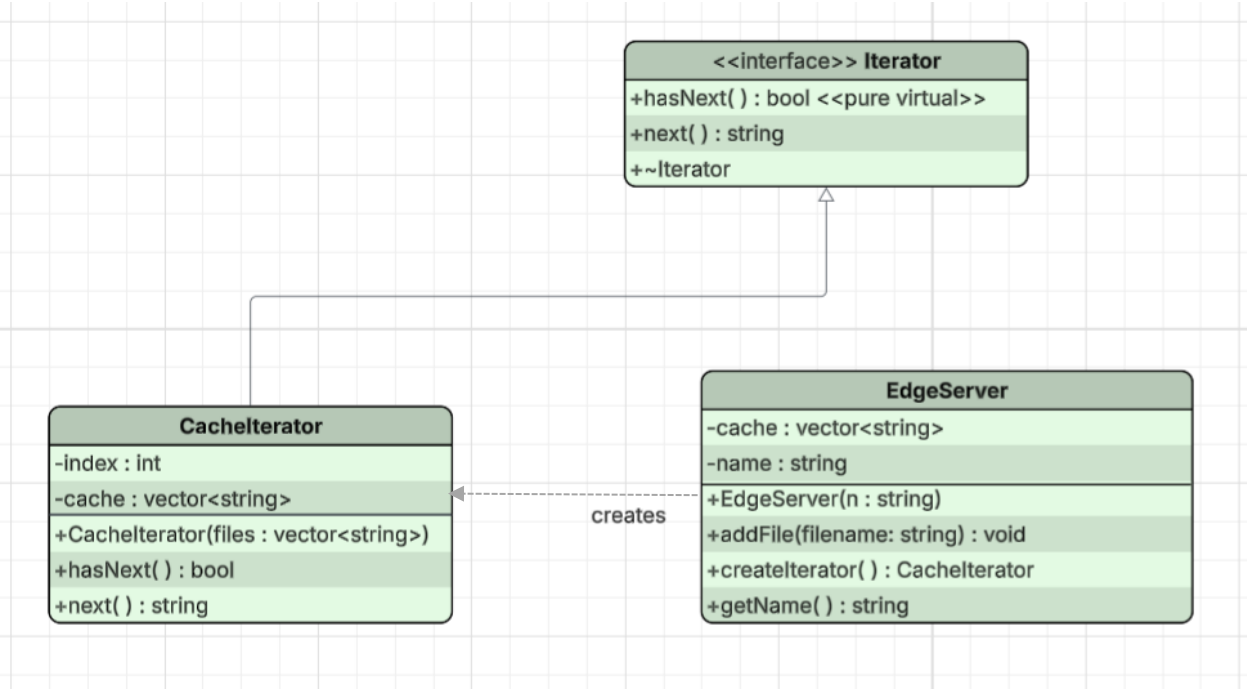
```
        cout << "File: " << it.next() << endl;
    }

    return 0;
}
```

Task D — UML Class Diagrams

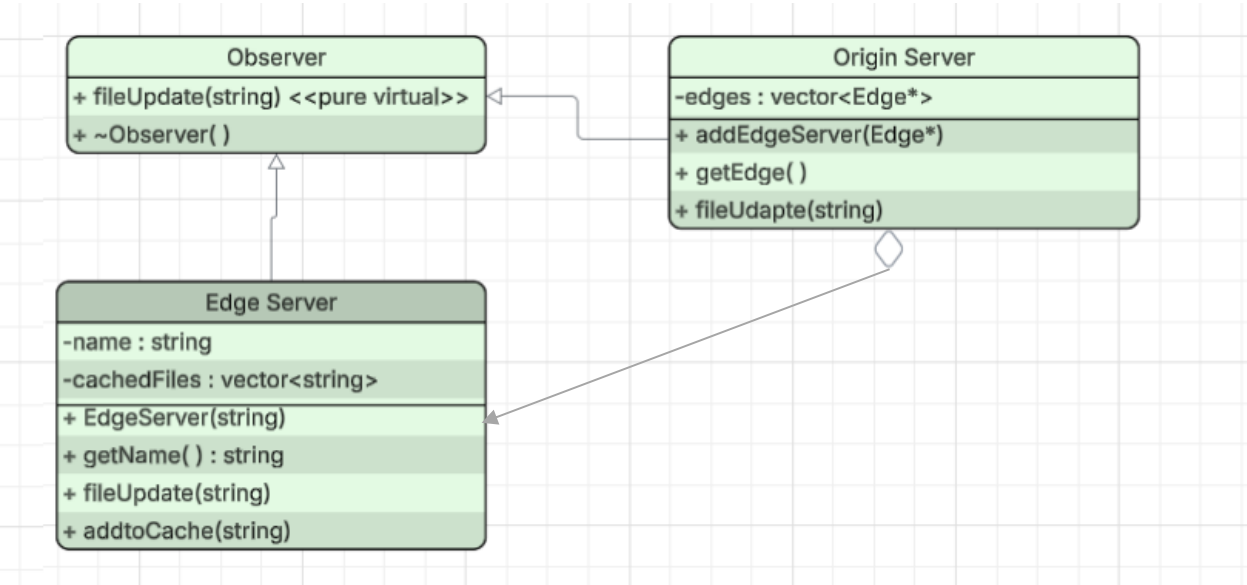
UML Diagram 1 — Proxy Pattern



UML Diagram 2 — Iterator Pattern



UML Diagram 3 — Observer Pattern



UML Diagram 3 — Strategy Pattern